

Universidade Tecnológica Federal do Paraná

Engenharia de Software - Curso de Arquitetura de Software (AS27S)

INSTRUTOR: Prof. Dr. Gustavo Santos

Aluno Juan Gabriel Benitez Bogo, RA 2601753

CCH - Design Patterns Template Methods

Link do projeto:

<https://github.com/juaNGb01/Trabalho-Arquitetura---Designs-Patterns.git>

Problema

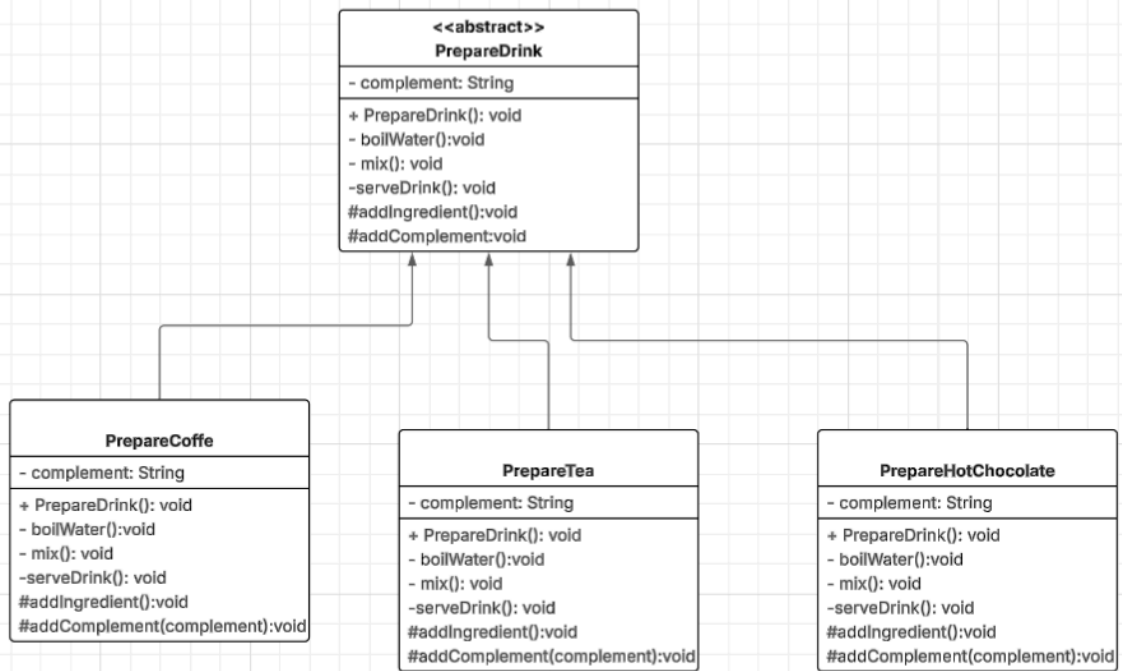
Como implementar um programa para o preparo de diferentes tipos de bebidas quentes (café, chá, chocolate quente), na qual, elas seguem o mesmo processo de preparo, e se diferenciam apenas pelos ingredientes utilizados

Descrição da Solução

A utilização do template method permitiu a criação de uma classe abstrata chamada `prepareDrink`, na qual tinha a função de criar todos os métodos para a preparação das bebidas, criando etapas bem definidas e padrão para serem seguidas durante o processo de preparação. Contudo mesmo com essa padronização ainda é possível criar bebidas únicas, com diferentes tipos e complementos que podem ser determinados de acordo com o gosto do usuário.

Para a criação dessas bebidas de forma diferente a classe `prepareDrink` implementa dois métodos abstratos (`addIngredient()` e `addComplement()`) que permitem que a bebida possa se adaptar de acordo com a classe que está sendo chamada

Visão Geral [exemplo]



Exemplo de Código java

Classe abstrata prepareDrink

Java

```
public abstract class PrepareDrink {

    private String complement;

    public PrepareDrink(String complement) {

        this.complement = complement;
    }
}
```

```
}

public final void prepareDrink() {

    System.out.println("=== Preparando bebida ===");

    boilWater();

    addIngredient();

    addComplements(complement);

    mix();

    serveDrink();

    System.out.println("=== Bebida pronta! ===");

}

private void boilWater() {

    System.out.println("Esquentando a água...");

}

private void mix() {

    System.out.println("Misturando bem...");

}

private void serveDrink() {

    System.out.println("Servindo na xícara");

}
```

```
// Métodos abstratos - cada bebida implementa do seu jeito

protected abstract void addIngredient();

protected abstract void addComplements(String complement);
}
```

Classe PrepareCoffe - uma das classe concretas que implementa a PrepareDrink

```
public class PrepareCoffe extends PrepareDrink {

    public PrepareCoffe(String complement) {

        super(complement);
    }

    @Override

    protected void addIngredient() {

        System.out.println("Adicionando Café");
    }

    @Override

    protected void addComplements(String complement) {

        System.out.println("Adicionando complemento: " + complement);
    }
}
```

-
- Consequências [vantagens e desvantagens]
 - As tarefas comuns durante os processos (como esquentar água ou misturar os ingredientes) são implementadas apenas uma vez na classe base - sem duplicação
 - Em casos de alteração do fluxo de preparo das bebidas a alteração é realizada apenas na classe abstrata e repassada para as outras
 - Criação de bebidas é muito mais simples, uma vez, que ele basta apenas criar uma nova classe concreta que implementa a classe abstrata
 - Como as classes concretas das bebidas dependem da classe abstrata temos um forte acoplamento pela hierarquia